# Docker:

## Application Containerization

# Agenda

- What is Docker?
- Docker Architecture
- Installing Docker
- Dockerfile
- Docker Images
- Docker and Microservices

# What is Docker ?

**Docker** separates applications from infrastructure using the container technology. This is similar to the way VMs separate operating systems from the hardware. Using Docker containers, you can deploy, replicate, move, and back up a workload even more quickly and easily than you can do so using virtual machines. Docker container are highly **scalable** (can be expanded rapidly), **portable** (Dockerized application are extremely portable). Docker uses **google** '**Go**' programming language for Docker development.

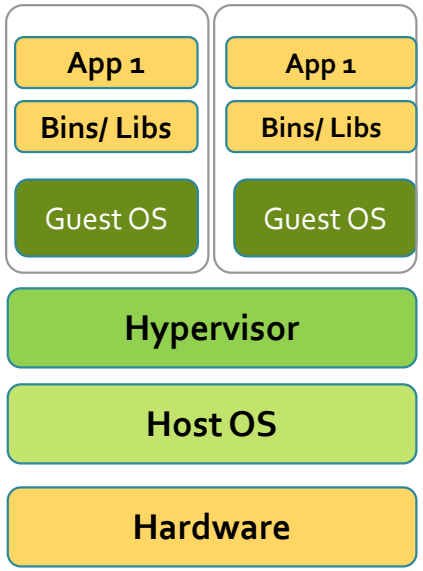
- **Docker Registries:**

A registry is a storage and content delivery system, holding named Docker images, available in different tagged versions. Users interact with a registry by using Docker push and pull commands.
One can create and share Docker image on 'Docker hub' for sharing with co-workers, customers etc.

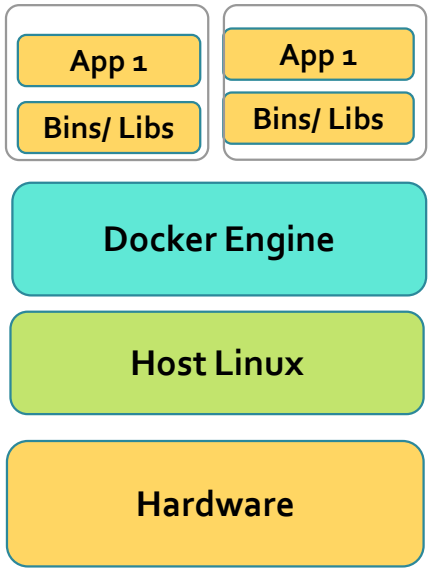
```
$ docker pull
$ docker run <container name>
```

- Docker formerly known as **dotCloud**

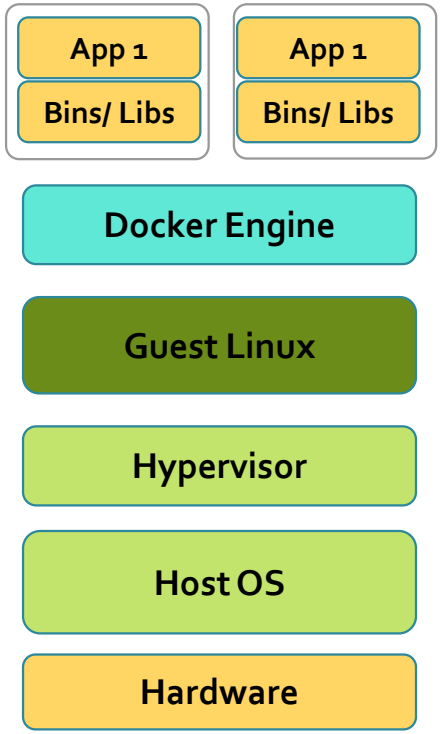
# Virtual Machine V/S Containers




Virtual Machine

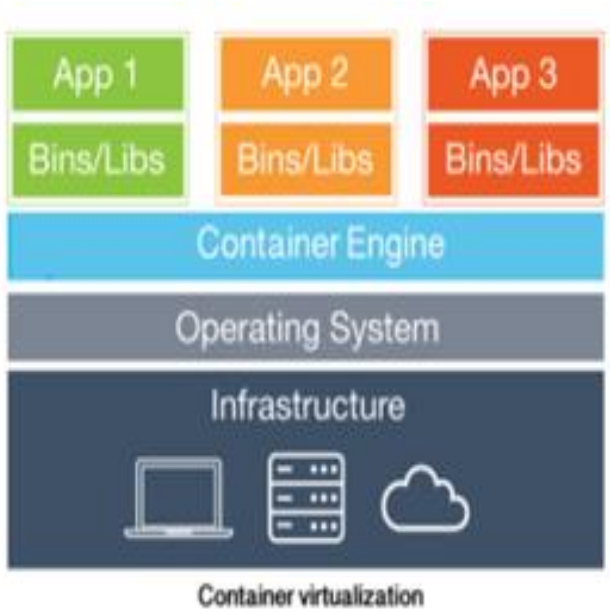


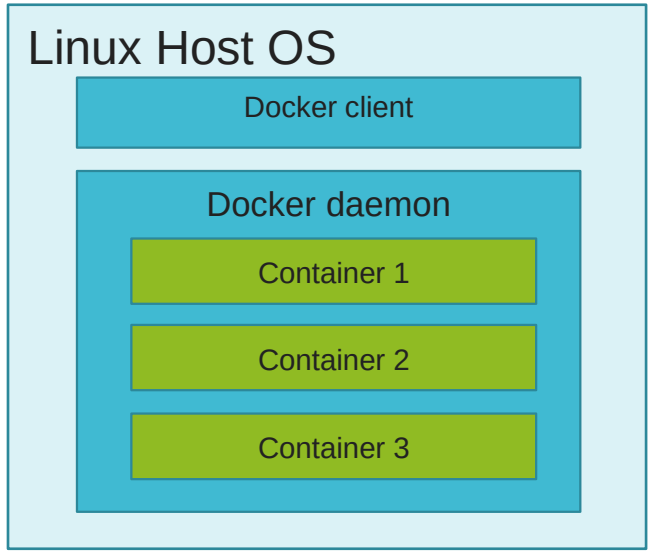

Docker on Linux / Win Server 2019 and later




Docker on Non-Linux (Windows / Mac)

# Docker
## Architecture




- Container are very light weight that shares same OS kernel, as compared to a VM.
- Container still provides isolated user space like a VM. This can also be referred as container virtualization or OS level virtualization.
- Container share Host OS resources like, RAM, CPU, Storage, network, etc.
- Each container has its own set of file system, like /etc, /var, /opt, etc.
- Container is created from a container image that can be customized. One can create a custom container image using Dockerfile that suits to your application requirements.
- Containerization is based on the Linux Kernel Namespaces.

# Installing Docker on Windows / Mac

**Docker** can be installed on **Windows** as well as Linux flavor machine.

- Docker can be installed on Windows desktop OS or MAC machine using **Docker-Desktop**
- Download Docker for windows from the URL,
- https://docs.docker.com/docker-for-windows/install
- https://docs.docker.com/desktop/install/mac-install/
- **Docker Desktop** on Windows needs **WSL-2** to be enabled.

```
$ docker version

Client: Docker Engine - Community
 Version:           24.0.4
 API version:       1.43
 Go version:        go1.20.5
 Git commit:        3713ee1
 Built:             Fri Jul  7 14:50:57 2023
 OS/Arch:           linux/amd64
 Context:           default
Server: Docker Engine - Community
 Engine:
  Version:          24.0.4
```

# Install Docker-CE
## on Linux

- To, install Docker on Linux / Ubuntu OS, follow the link mentioned below.
  - https://docs.docker.com/engine/install/ubuntu/
- For other Operating systems, follow the link below,
  - https://docs.docker.com/engine/install/

# Docker commands

**Now** to use a docker **image** that is part of the docker repository, use below command,

**Docker** promotes the **standardization** ideology and so that a container created on a local machine can easily be **ported** to EC2, Azure or any other **cloud** environment.

`$ docker run -it ubuntu /bin/bash` ---- this tells the docker to run in interactive mode and assign a terminal, use the **Ubuntu image** for the container to create and then run the /bin/bash command once the container is ready.

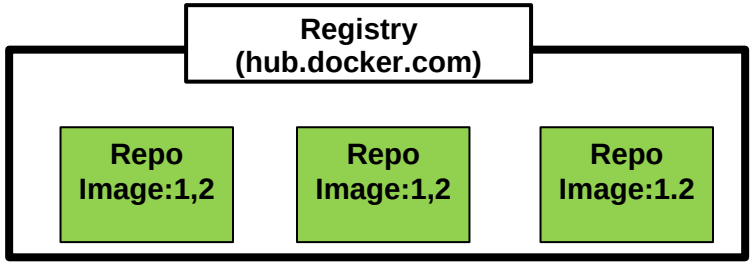
```
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
ffa920a196d9: Pull complete
5e951a8003f1: Pull complete
1764a46f680f: Pull complete
31c87eb8694d: Pull complete
1d6f52380bd7: Pull complete
fa454a3d0892: Pull complete
Digest: sha256:0c01b5105fc57b5eb8b7bb1d697b9dc5602022f69a0a33bf282b117e7a754a91
Status: Downloaded newer image for ubuntu:latest
```

`$ docker ps -a` --- this will show all processes running on the host.

`$ docker start <container code>` --- start the container

`$ docker attach <container code>` --- attach to running container

**Docker HUB: Registries and Repository:**



# Docker Commands

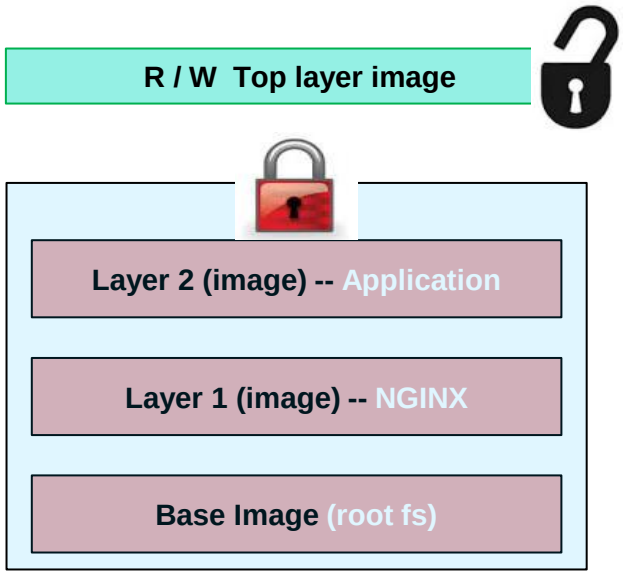
**Docker** provides commands to manage, to create and share the images and Containers.

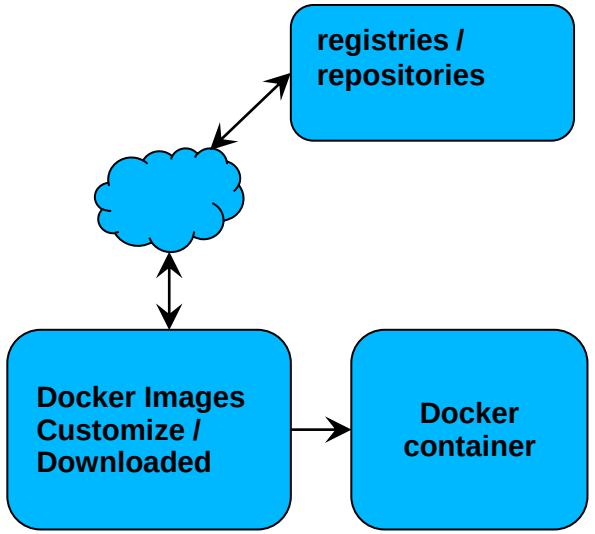
| Commands: | |
|---|---|
| attach | Attach local standard input, output, and error streams to a running containe |
| commit | Create a new image from a container's changes |
| cp | Copy files/folders between a container and the local filesystem |
| create | Create a new container |
| diff | Inspect changes to files or directories on a container's filesystem |
| events | Get real time events from the server |
| export | Export a container's filesystem as a tar archive |
| history | Show the history of an image |
| import | Import the contents from a tarball to create a filesystem image |
| inspect | Return low-level information on Docker objects |
| kill | Kill one or more running containers |
| load | Load an image from a tar archive or STDIN |
| logs | Fetch the logs of a container |
| pause | Pause all processes within one or more containers |
| port | List port mappings or a specific mapping for the container |
| rename | Rename a container |
| restart | Restart one or more containers |
| rm | Remove one or more containers |
| rmi | Remove one or more images |
| save | Save one or more images to a tar archive (streamed to STDOUT by default) |
| start | Start one or more stopped containers |
| stats | Display a live stream of container(s) resource usage statistics |
| stop | Stop one or more running containers |
| tag | Create a tag TARGET_IMAGE that refers to SOURCE_IMAGE |
| top | Display the running processes of a container |
| unpause | Unpause all processes within one or more containers |
| update | Update configuration of one or more containers |
| wait | Block until one or more containers stop, then print their exit codes |

# Docker Images

**Docker Images**: Docker images can be **stacked** on one another to form a single image. This is achieved using the **Union mounts.**

This scenario is very much likely when we go on adding new application on the existing images. This provides a lot of flexibility of maintaining and updating, making it highly scalable and portable.



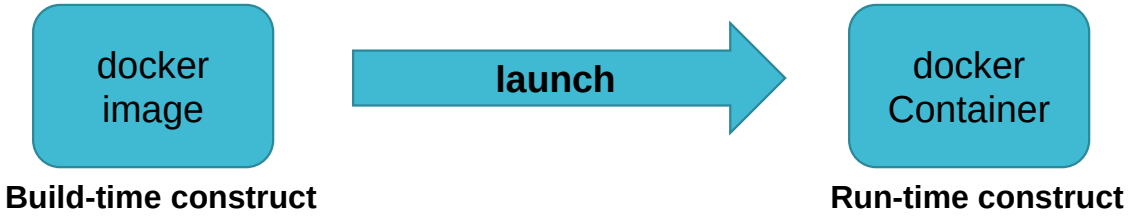



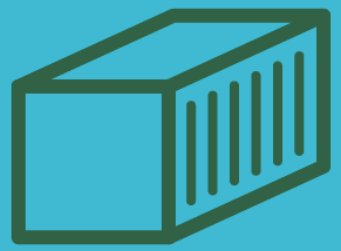
# Docker Images

When we pull docker images from hub / repository, there are other images that are pulled along with it. Those are the **layers** of images that are part of the top level image.

| REPOSITORY | TAG | IMAGE ID | CREATED | VIRTUAL SIZE |
|---|---|---|---|---|
| ubuntu | latest | fa454a3d0892 | 3 weeks ago | 129.5 MB |
| <none> | <none> | 1d6f52380bd7 | 3 weeks ago | 129.5 MB |
| <none> | <none> | 31c87eb8694d | 3 weeks ago | 129.5 MB |
| <none> | <none> | 1764a46f680f | 3 weeks ago | 129.5 MB |
| <none> | <none> | 5e951a8003f1 | 3 weeks ago | 129.5 MB |
| <none> | <none> | ffa920a196d9 | 3 weeks ago | 129.5 MB |
| centos | latest | 2785d012ae3e | 9 weeks ago | 191.8 MB |
| <none> | <none> | 8a478b6da50a | 9 weeks ago | 191.8 MB |
| <none> | <none> | a06898bd69f9 | 9 weeks ago | 191.8 MB |
| <none> | <none> | 8aae2253a786 | 5 months ago | 0 B |

`$ docker pull –a <image_name>` … pulls all docker images for an image name e.g. docker pull –a ubuntu.. Will pull all available versions of Ubuntu images from docker hub.





# Docker Containers

Every Docker container get a **writable top layer** where changes to the container are done, like installing a new app, file creation, IP address changes etc.,

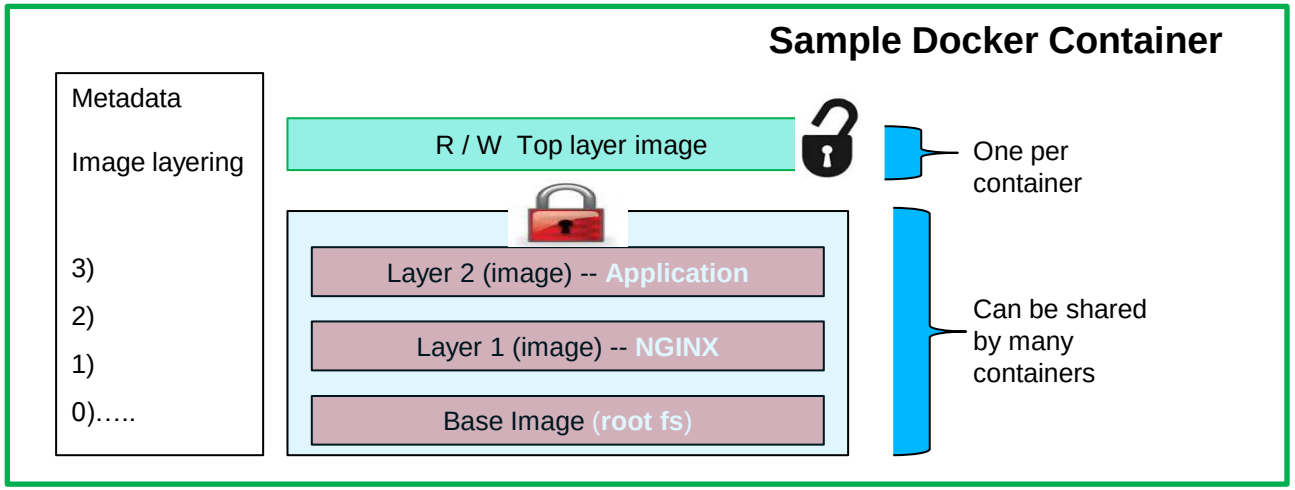
`$ docker run -it ubuntu /bin/bash` --- run a container using mentioned image, unpacks the image, layers and build a container with a terminal in interactive mode and assign a shell prompt.

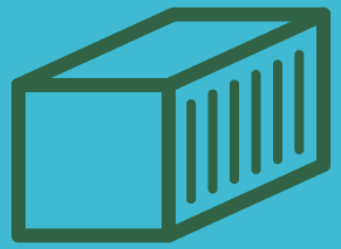
`$ docker run -dt ubuntu` ----- run a docker container in detached mode. The container can be attached to with the command,

```
$ docker attach <container_ID>
```

Also one get attached to docker running container using command,

```
$ docker exec -it <docker_container_id> bash
```





# Docker Containers

Inside `/var/lib/docker/container` folder are the container structures.

```
root@dockerhost:/var/lib/docker/containers# tree
.
├── a28b32ef431bab576f7108a3891a4c0254b42b1582a1173bbb923c4a4bd7e438
│   ├── a28b32ef431bab576f7108a3891a4c0254b42b1582a1173bbb923c4a4bd7e438-json.log
│   ├── config.json
│   ├── hostconfig.json
│   ├── hostname
│   ├── hosts
│   ├── mqueue
│   ├── resolv.conf
│   ├── resolv.conf.hash
│   └── shm
└── e709c244a36ce0ea6786b66b5322293a0d70f96bb5d66af866f1b8351c687a40
    ├── config.json
    ├── e709c244a36ce0ea6786b66b5322293a0d70f96bb5d66af866f1b8351c687a40-json.log
    ├── hostconfig.json
    ├── hostname
    ├── hosts
    ├── mqueue
    ├── resolv.conf
    ├── resolv.conf.hash
    └── shm
```

We can create images that can be exported / shared with using the command,

```
$ docker commit <container ID> <image_name>
```

```
$ docker ps -a
```

--- to list all docker container that's been running currently and in the past.

```
$ docker commit e709c244a36c new_img
```

The image that gets created is only the top layer image with very small size that holds only the latest updates, etc.

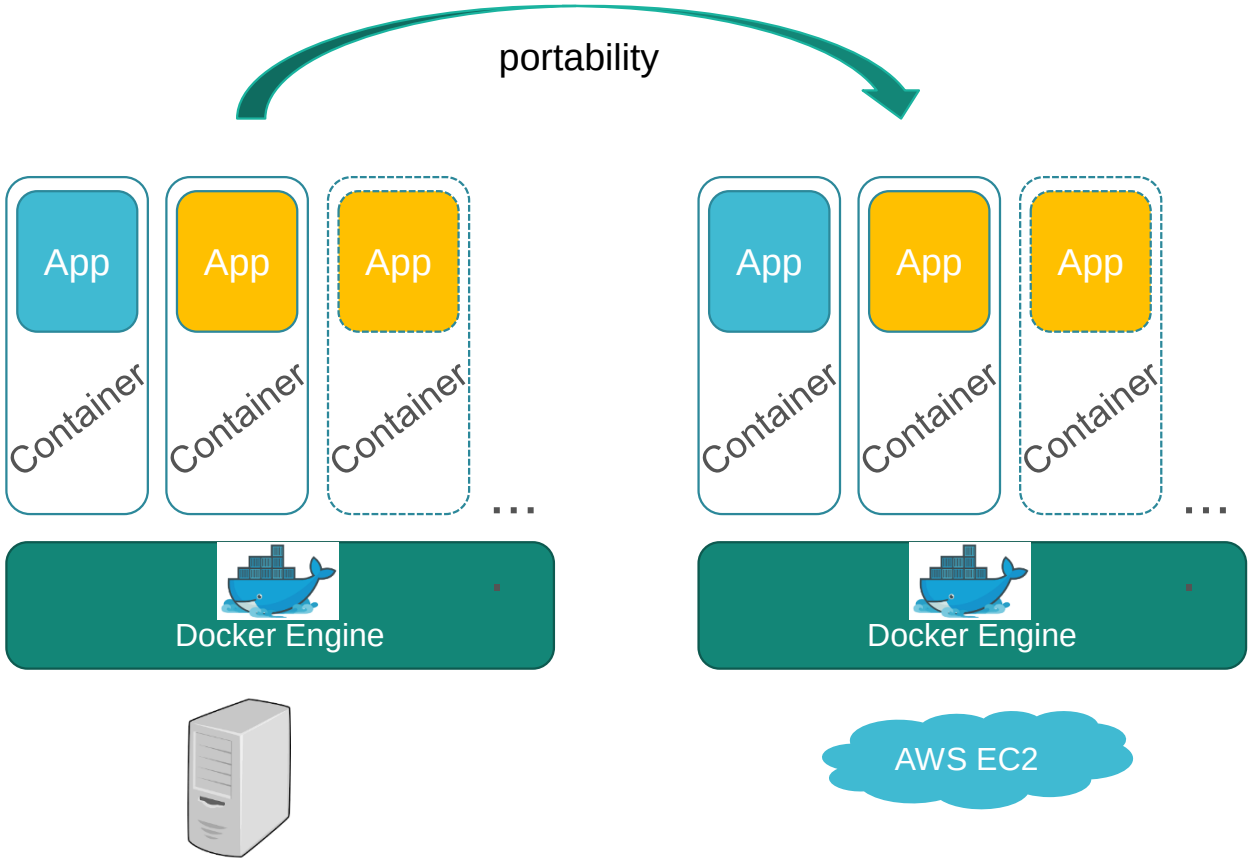
# Networking containers

- In Docker environment on the docker host, there's always a default bridge network. Services from the same applications run inside one common network. For any new applications to start on the host, we create a separate new network and run apps / services inside the new network.
- We can list all networks on the host by using  command,
- `$ docker network ls`

```
NETWORK ID     NAME      DRIVER    SCOPE
f71a95282ed0   bridge    bridge    local
cc1302596540   host      host      local
011a0c94a404   none      null      local
```

- To create a new custom network bridge interface, we can use command,
- `$ docker network create -d bridge mynetwork`
- To create a container inside a specific network, use command,
- `$ docker run –dt --net=mynetwork --name=generic ubuntu:latest`

# Application
## Containerization



# DockerFile

**Dockerfile**: Declarative way to construct an image.

- A **Dockerfile** is similar in concept to the recipes and manifests found in configuration management tools like Chef or Puppet.
- **Dockerfile** is much more stripped down than the IA tools, consisting of a single file with a DSL that has a handful of instructions.
- Below is sample **Dockerfile** that uses the ubuntu image, **RUN** command is used to run a certain command inside a container., **ADD** is used to add a file from host machine to the container and place it at desired location.

```
FROM ubuntu:latest
RUN apt-get update
RUN apt-get install -y python
python-pip wget
RUN pip install Flask
WORKDIR /home
```

- Images are build time constructs and Container are run time constructs.

```
$ docker build –t <dockerimage_name> <dockerfile_path>
```

# Docker registries and repos

https://registry.hub.docker.com --- docker official registry

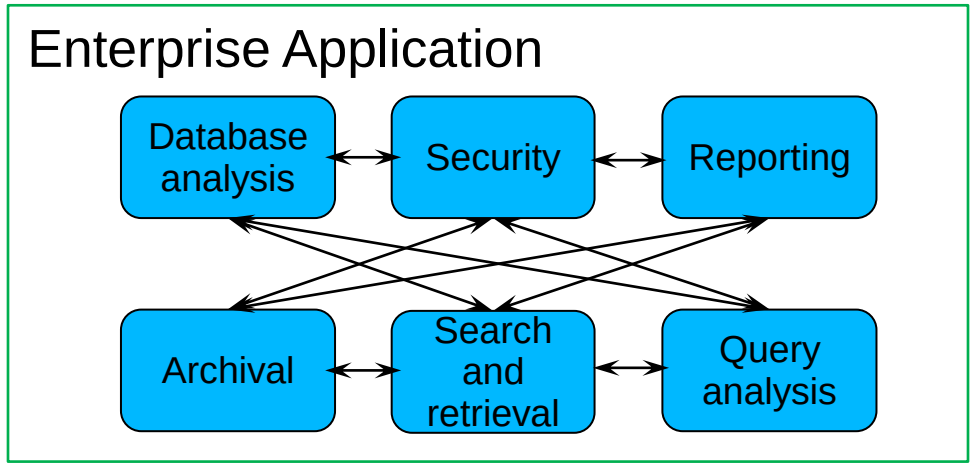
https://hub.docker.com/ --- create account on docker hub to upload / contribute to the docker public repository.

https://store.docker.com --- image repositories.

Use `$ docker pull <image_name>…` command to pull images from registries.

# Docker and Microservices

In an application there are multiple services and such services can be offered in an Docker container as a service and such container working together forms the entire application. This way, the individual service can be upgraded / maintained without imposing any downtime on the application.



# Thank You